

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

class Linear_Regression():
    def __init__(self, learning_rate, no_of_iterations):
        self.learning_rate = learning_rate
        self.no_of_iterations = no_of_iterations

    def fit(self, X, Y):

        self.m, self.n = X.shape
        self.w = np.zeros(self.n)
        self.b = 0
        self.X = X
        self.Y = Y

        for i in range(self.no_of_iterations):
            self.update_weights()

    def update_weights(self):
        Y_prediction = self.predict(self.X)

        dw = - (2 * (self.X.T).dot(self.Y - Y_prediction)) / self.m
        db = - 2 * np.sum(self.Y - Y_prediction) / self.m

        self.w = self.w - self.learning_rate * dw
        self.b = self.b - self.learning_rate * db

    def predict(self, X):
        return X.dot(self.w) + self.b
```

[12] ✓ 0.0s Python

```
chirps_per_min = np.array([
    20.0, 16.0, 19.8, 18.4, 17.1, 15.5, 14.7, 17.1,
    15.4, 16.2, 15.0, 17.2, 16.0, 17.0, 14.4
])
```

[13] ✓ 0.1s Python

nd 42 cells to analyze

Spaces: 4 LF Go Live Connect to MSSQL

```
chirps_per_min = np.array([
    20.0, 16.0, 19.8, 18.4, 17.1, 15.5, 14.7, 17.1,
    15.4, 16.2, 15.0, 17.2, 16.0, 17.0, 14.4
])

temperature_F = np.array([
    88.6, 71.6, 93.3, 84.3, 80.6, 75.2, 69.7, 82.0,
    69.4, 83.3, 79.6, 82.6, 80.6, 83.5, 76.3
])

df = pd.DataFrame({'chirps_per_min': chirps_per_min, 'temperature_F': temperature_F})
df.head()
```

[13] ✓ 0.1s Python

	chirps_per_min	temperature_F
0	20.0	88.6
1	16.0	71.6
2	19.8	93.3
3	18.4	84.3
4	17.1	80.6

```
print('Shape:', df.shape)
print()
print(df.describe().round(2))
```

[14] ✓ 0.0s Python

Shape: (15, 2)

	chirps_per_min	temperature_F
count	15.00	15.00
mean	16.65	80.04
std	1.70	6.71
min	14.40	69.40
25%	15.45	75.75
50%	16.20	80.60
75%	17.15	83.40
max	20.00	93.30

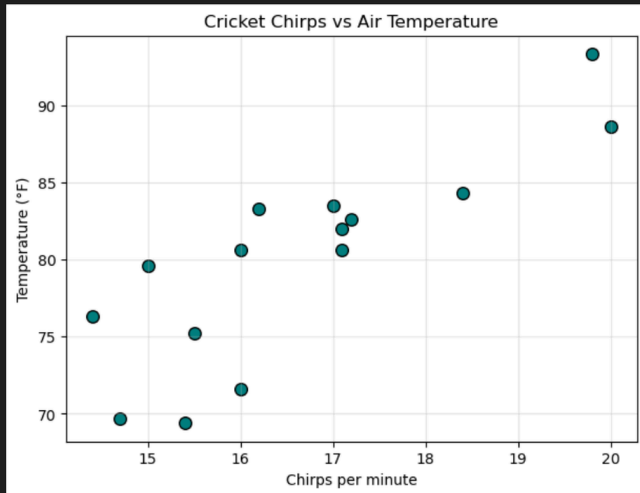
Exploratory Data Analysis

```
plt.figure(figsize=(7, 5))
plt.scatter(df['chirps_per_min'], df['temperature_F'], color='teal', edgecolor='k', s=70)
plt.xlabel('Chirps per minute')
plt.ylabel('Temperature (°F)')
plt.title('Cricket Chirps vs Air Temperature')
plt.grid(alpha=0.3)
plt.show()
```

```
corr = df['chirps_per_min'].corr(df['temperature_F'])
print(f'Pearson correlation = {corr:.3f} -> strong positive linear relation')
```

✓ 0.2s

Python



Pearson correlation = 0.835 -> strong positive linear relation

Train / Test Split

```
X = df[['chirps_per_min']].values
y = df[['temperature_F']].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=88)

print('Train size:', len(X_train))
print('Test size:', len(X_test))
```

✓ 0.0s

Python

Train size: 10
Test size: 5

```
X_mean = X_train.mean()
X_std = X_train.std()

X_train_norm = (X_train - X_mean) / X_std
X_test_norm = (X_test - X_mean) / X_std

print(f'Train mean = {X_mean:.2f}, std = {X_std:.2f}')
```

✓ 0.0s

Python

Train mean = 16.54, std = 1.36

Train the Model (gradient descent)

```
model = LinearRegression(learning_rate=0.05, no_of_iterations=2000)
model.fit(X_train_norm, y_train)

print(f'Weight (normalised space) : {model.w[0]:.3f}')
print(f'Bias (normalised space) : {model.b:.3f}')
```

✓ 0.0s

Python

Weight (normalised space) : 3.997
Bias (normalised space) : 79.500

Convert Weights Back to Original Units

The model was fitted on standardised X , so we map it back: $y = w * (x - \text{mean})/\text{std} + b \rightarrow y = (w/\text{std}) * x + (b - w*\text{mean}/\text{std})$.

```

slope      = model.w[0] / X_std
intercept  = model.b - model.w[0] * X_mean / X_std

print(f'Slope      (m): {slope:.3f}')
print(f'Intercept (c): {intercept:.3f}')
print(f'Model: Temp = {slope:.3f} * chirps_per_min + {intercept:.3f}')
```

[19]

✓ 0.0s

Python

```

... Slope      (m): 2.948
Intercept (c): 30.737
Model: Temp = 2.948 * chirps_per_min + 30.737
```

Evaluate on the Test Set

```

y_pred = model.predict(X_test_norm)

r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f'R^2 score = {r2:.3f}')
print(f'RMSE      = {rmse:.3f} °F')
print()
```

[20]

✓ 0.0s

Python

```

... R^2 score = 0.848
RMSE      = 3.084 °F
```

	chirps_per_min	actual_temp	predicted_temp
0	18.4	84.3	85.0
1	14.4	76.3	73.2
2	14.7	69.7	74.1
3	17.1	82.0	81.2
4	19.8	93.3	89.1

Visualise the Fit

Generate

+ Code

+ Markdown

```

x_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
x_line_norm = (x_line - X_mean) / X_std
y_line = model.predict(x_line_norm)

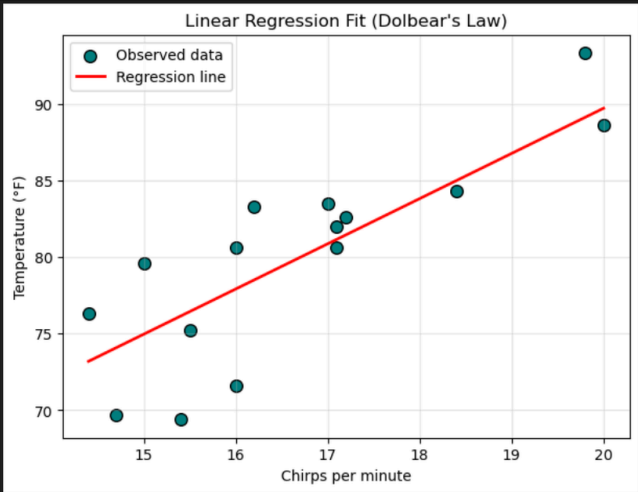
plt.figure(figsize=(7, 5))
plt.scatter(X, y, color='teal', edgecolor='k', s=70, label='Observed data')
plt.plot(x_line, y_line, color='red', lw=2, label='Regression line')
plt.xlabel('Chirps per minute')
plt.ylabel('Temperature (°F)')
plt.title("Linear Regression Fit (Dolbear's Law)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[1]

✓ 0.0s

Python





Make a Prediction

```
chirp_input = np.array([[18.0]])
chirp_input_norm = (chirp_input - X_mean) / X_std
predicted_temp = model.predict(chirp_input_norm)[0]
print(f'If a cricket chirps {chirp_input[0][0]:.0f} times/min, predicted temp = {predicted_temp:.1f} °F')
```

✓ 0.0s

Python

If a cricket chirps 18 times/min, predicted temp = 83.8 °F

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score
```

```
years = np.array(list(range(1959, 2025)))
```

```
co2_ppm = np.array([
    315.97, 316.91, 317.64, 318.45, 318.99, 319.62, 320.04, 321.38, 322.16, 323.04,
    324.62, 325.68, 326.32, 327.45, 329.68, 330.18, 331.08, 332.05, 333.78, 335.41,
    336.78, 338.68, 339.93, 341.13, 342.78, 344.42, 345.90, 347.16, 348.93, 351.48,
    352.91, 354.19, 355.62, 356.45, 357.10, 358.83, 360.82, 362.61, 363.73, 366.70,
    368.38, 369.55, 371.14, 373.28, 375.80, 377.52, 379.80, 381.90, 383.79, 385.60,
    387.43, 389.90, 391.65, 393.85, 396.52, 398.65, 400.83, 404.24, 406.55, 408.52,
    411.44, 414.24, 416.45, 418.56, 421.08, 424.61
])
```

```
df = pd.DataFrame({'year': years, 'co2_ppm': co2_ppm})
df.head()
```

✓ 0.3s

Python

	year	co2_ppm
0	1959	315.97
1	1960	316.91
2	1961	317.64
3	1962	318.45
4	1963	318.99

```
print('Shape:', df.shape)
print()
print(df.describe().round(2))
```

✓ 0.1s

Python

Shape: (66, 2)

Exploratory Data Analysis

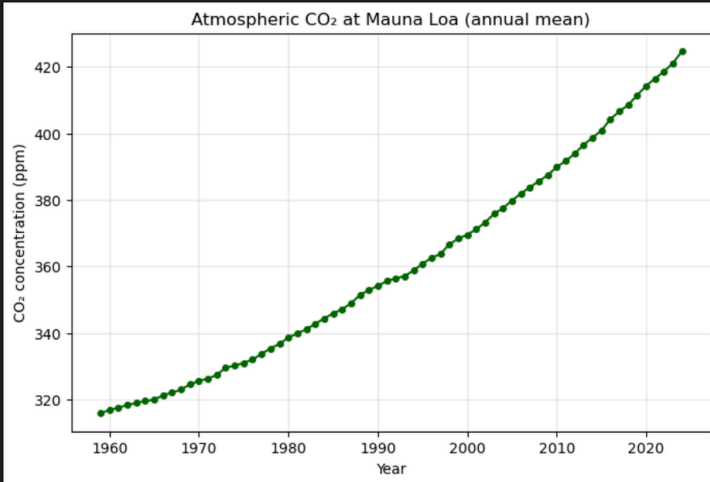
/MachineLearningFrom0/datasets

```
plt.figure(figsize=(8, 5))
plt.plot(df['year'], df['co2_ppm'], 'o-', color='darkgreen', markersize=4)
plt.xlabel('Year')
plt.ylabel('CO2 concentration (ppm)')
plt.title('Atmospheric CO2 at Mauna Loa (annual mean)')
plt.grid(alpha=0.3)
plt.show()

print('The curve bends upward over time -> a straight line would be a poor fit.')
```

11] ✓ 0.1s

Python



12] The curve bends upward over time -> a straight line would be a poor fit.

✓ Train / Test Split (chronological)

```
X = df[['year']].values
y = df['co2_ppm'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=False
)

print('Train:', int(X_train.min()), '-', int(X_train.max()))
print('Test :', int(X_test.min()), '-', int(X_test.max()))
```

12] ✓ 0.0s

Python

Train: 1959 - 2010
Test : 2011 - 2024

Try Polynomials of Different Degrees (1 to 4)

```
results = {}
for degree in range(1, 5):
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    results[degree] = {
        'model': model,
        'r2': r2_score(y_test, y_pred),
        'rmse': np.sqrt(mean_squared_error(y_test, y_pred)),
    }

print('Degree | R^2      | RMSE (ppm)')
print('-' * 35)
for degree, r in results.items():
    print(f' {degree} | {r["r2"]:.4f} | {r["rmse"]:.3f}')
```

13] ✓ 0.0s

Python

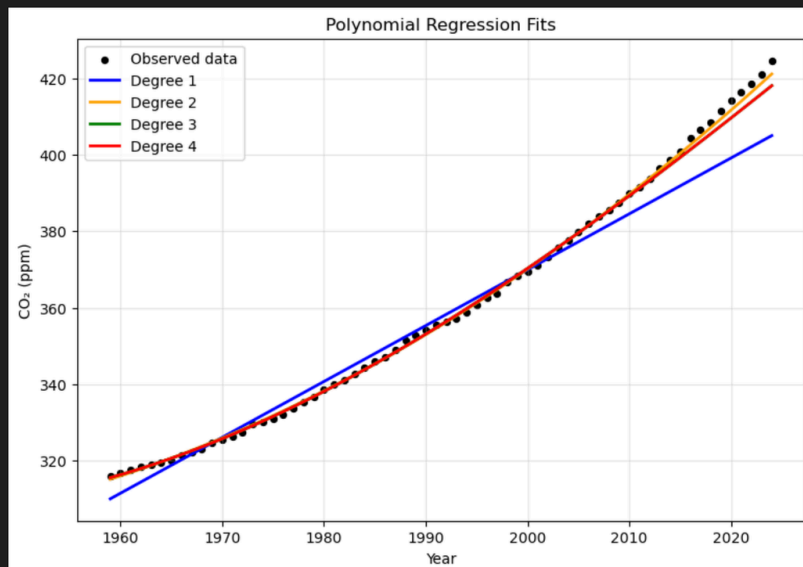
Degree	R ²	RMSE (ppm)
1	-0.6179	12.872
2	0.9660	1.867
3	0.8704	3.643
4	0.8701	3.647

```
plt.figure(figsize=(9, 6))
plt.scatter(X, y, color='black', s=18, label='Observed data')

colors = {1: 'blue', 2: 'orange', 3: 'green', 4: 'red'}
x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)

for degree, r in results.items():
    y_line = r['model'].predict(x_line)
    plt.plot(x_line, y_line, color=colors[degree], lw=2, label=f'Degree {degree}')

plt.xlabel('Year')
plt.ylabel('CO2 (ppm)')
plt.title('Polynomial Regression Fits')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Inspect the Best Model's Coefficients

```
best_degree = max(results, key=lambda d: results[d]['r2'])
best = results[best_degree]['model']

poly = best.named_steps['polynomialfeatures']
lin = best.named_steps['linearregression']

print(f'Best degree : {poly.degree}')
print(f'Intercept : {lin.intercept_:.3f}')
print('Coefficients (lowest to highest power):')
for p, coef in enumerate(lin.coef_):
    print(f'x^{p}: {coef:.6f}')
```

```
Best degree : 2
Intercept : 44982.287
Coefficients (lowest to highest power):
x^0: 0.000000
x^1: -46.446405
x^2: 0.012070
```

Predict Future CO₂ Levels

```
future_years = np.array([[2030], [2040], [2050]])
predictions = best.predict(future_years)

for yr, p in zip(future_years.flatten(), predictions):
    print(f'Predicted CO2 in {yr}: {p:.1f} ppm')
```

```
Predicted CO2 in 2030: 436.0 ppm
Predicted CO2 in 2040: 462.8 ppm
Predicted CO2 in 2050: 492.0 ppm
```